0

(Task 1 of 15) Welcome to SMoL Tutor! This mini tutorial lets you review topics from other tutorials or get a quick sense of what the Tutor offers.

1

(Task 2 of 15) Let's start with two simple questions to get you familiar with the question formats.

2

(Task 3 of 15) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x 1)
(defvar y (+ x 2))
x
y
```

Python

```
x = 1
y = x + 2
print(x)
print(y)
```

3

`1 3`

4

You predicted the output correctly 🎉

The first definition binds `x` to the value `1`. The second definition binds `y` to the value of `(+ x 2)`, which is 3. So, the result of this program is `1 3`.

Click here to run this program in the Stacker.

5

(Task 4 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program.

```
(defvar x 2)
(defvar y 3)
(defvar z (+ x y))
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

6

✅
```
(defvar z (+ x y))
(defvar x 2)
(defvar y 3)
```

☐
```
(defvar x 2)
(defvar y 3)
(defvar z 5)
```

☐
```
(defvar x 2)
(defvar y (+ x 1))
```

```
(defvar y (+ x 1))
(defvar z (+ x y))
```

☐
```
(defvar x 2)
(defvar y 3)
(defvar tmp (+ x y))
(defvar z tmp)
```

You gave the correct answer 🎉 7

(Task 5 of 15) Now, let's dive into the real questions! 8

Lispy |

(Task 6 of 15) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x 12)
(deffun (f)
  x)
(deffun (g)
  (set! x 0)
  (f))
(g)
(set! x 1)
(f)
```

JavaScript

```
let x = 12;
function f() {
  return x;
}
function g() {
  x = 0;
  return f();
}
console.log(g());
x = 1;
console.log(f());
```

`0 1` 10

You predicted the output correctly 🎉 11

There is exactly one variable `x`. The `x` in `(set! x 0)` refers to that variable. Calling `f` evaluates `(set! x 0)`, which mutates `x`. When the value of `x` is eventually printed, we see the new value.

Click here to run this program in the Stacker.

Lispy |

(Task 7 of 15) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x (mvec 19 73 28))
(defvar y (mvec x x))
(vec-set! (vec-ref y 0) 0 64)
(vec-ref x 1)
```

Pseudo

```
let x = vec[19, 73, 28]
let y = vec[x, x]
y[0][0] = 64
print(x[1])
```

```
(vec ref y 1)                    print(y[1])
```

```
#(64 73 28)
```

13

You predicted the output correctly 🎉

14

This program first creates a three-element vector and binds it to `x`. After that, the program creates a two-element vector. Both elements refer to the first vector. After that, the `0`-th element of the three-element vector is replaced with `64`. Finally, the three-element vector is printed.

Click here to run this program in the Stacker.

15

(Task 8 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program.

```
(defvar x n)
(defvar y (mvec n 89 12))
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

16

☐
```
(defvar x n)
(defvar y (mvec x 89 12))
```

☐
```
(defvar tmp n)
(defvar x tmp)
(defvar y (mvec tmp 89 12))
```

17

You gave the correct answer 🎉

18

(Task 9 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program.

```
(defvar x (mvec 19 73 28))
(defvar y (mvec (mvec 19 73 28) (mvec 19 73 28)))
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

19

✅
```
(defvar x (mvec 19 73 28))
(defvar y (mvec x x))
```

✅
```
(defvar x (mvec 19 73 28))
```

```
(defvar y (mvec x (mvec 19 73 28)))
```

✅
```
(defvar tmp (mvec 19 73 28))
(defvar x tmp)
(defvar y (mvec tmp tmp))
```

☐
```
(defvar x (mvec 19 73 28))
(defvar y (mvec (mvec (vec-ref x 0) (vec-ref x 1) (vec-ref x 2)) (m
```

You gave the correct answer 🎉 20

21

(Task 10 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program.

```
(defvar x n)
(defvar y n)
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

22

☐
```
(defvar x n)
(defvar y x)
```

☐
```
(defvar tmp n)
(defvar y tmp)
(defvar x tmp)
```

You gave the correct answer 🎉 23

24

(Task 11 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program.

```
(defvar x n)
(defvar y (f n))
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

25

☐
```
(defvar x n)
(defvar y (f x))
```

☐
```
(defvar tmp n)
(defvar x tmp)
```

```
(defvar y (f tmp))
```

✅

```
(defvar y (f x))
(defvar x n)
```

You gave the correct answer 🎉🎉 26

Lispy | 🎲

(Task 12 of 15) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x (mvec 55))
(defvar v (mvec x 55 55))
(set! x (mvec 66))
v
```

Scala 3

```
var x = Buffer(55)
var v = Buffer(x, 55, 55)
x = Buffer(66)
println(v)
```

`#(#(66) 55 55)` 28

Please briefly explain why you think the answer is `#(#(66) 55 55)`. 29

n 30

The answer is `#(#(55) 55 55)`. 31

▸ Textual explanation
▸ Graphical explanation

Lispy | 🎲

What is the result of running this program?

Lispy [Run ▶]

```
(defvar x (mvec 0))
(defvar v (mvec 2 x 3))
(set! x (mvec 1))
v
```

Pseudo

```
let x = vec[0]
let v = vec[2, x, 3]
x = vec[1]
print(v)
```

`#(2 #(0) 3)` 33

You predicted the output correctly 🎉🎉 34

(Task 13 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program. 35

```
(defvar x (mvec 10 48 95))
```

```
(defvar y (f (mvec 10 48 95)))
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

36

- ✅
```
(defvar x (mvec 10 48 95))
(defvar y (f x))
```
- ✅
```
(defvar tmp (mvec 10 48 95))
(defvar x tmp)
(defvar y (f tmp))
```
- ☐
```
(defvar x (mvec 10 48 95))
(defvar y (f (mvec (vec-ref x 0) (vec-ref x 1) (vec-ref x 2))))
```

37

You gave the correct answer 🎉

38

(Task 14 of 15) Imagine that you and your colleagues are given a large codebase. One of your colleagues would like to make a change to the following lines in a program.

```
(defvar x (mvec 52 70 61))
(defvar y (mvec 52 70 61))
```

Which (if any) of the following edits might break the code? (You can assume the variable `tmp` is never used elsewhere in the program.)

39

- ✅
```
(defvar x (mvec 52 70 61))
(defvar y x)
```
- ✅
```
(defvar tmp (mvec 52 70 61))
(defvar x tmp)
(defvar y tmp)
```
- ☐
```
(defvar x (mvec 52 70 61))
(defvar y (mvec (vec-ref x 0) (vec-ref x 1) (vec-ref x 2)))
```

40

You gave the correct answer 🎉

Lispy | 🎲

(Task 15 of 15) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x 1)
(deffun (f)
```

JavaScript

```
let x = 1;
function f() {
```

```
  x)                return x;
(deffun (g)       }
  (defvar x 2)    function g() {
  (f))              let x = 2;
                    return f();
(g)               }
                  console.log(g());
```

`1` 42

Please briefly explain why you think the answer is `1`. 43

b 44

You predicted the output correctly 🎉 45

`(g)` evaluates to `(f)`, which evaluates to `1` because `f` and `x` are both defined in the same block, and `f` does *not* get the `x` defined inside `g`.

Click here to run this program in the Stacker.

You have finished this tutorial 🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781423949222